

Giano: dove siamo

Che cosa c'è già, che cosa cambia col pagAmico, e quanto lavoro è davvero

Il quadro in venti righe

Giano oggi parla con **una sola marca** di cassa automatica: il cash recycler VNE. Si registra dalla stessa pagina di amministrazione con cui si registrano stampanti e cassette, e si configura con poco più di un indirizzo IP. Il dialogo è HTTP: Giano manda una richiesta, la macchina risponde, e ogni domanda è indipendente dalla precedente — non c'è connessione da tenere aperta, non c'è stato da gestire.

L'operatore ci arriva da **un solo punto dell'applicazione**: l'Asporto Espresso. Nessun'altra parte del programma — tavoli, post-vendita, palmare, registratore di cassa — sa che la cassa automatica esiste. Quando l'operatore finalizza un ordine in contanti e c'è almeno una macchina registrata e in uso, Giano apre una finestra modale senza pulsante di chiusura: da lì il cliente inserisce il denaro, la finestra mostra quanto ha messo e quanto gli tornerà indietro, e sotto un thread interroga la macchina ogni mezzo secondo per sapere a che punto è.

Del protocollo VNE sono implementati ventinove comandi, ma nel flusso di vendita ne vivono **cinque**. Tutto il resto — prelievi, svuotamenti, storici, configurazione — è codice senza chiamanti, scritto e mai collegato.

Il punto che conta per capire dove finisce il denaro: **dalla finestra di pagamento esce un solo bit di informazione**, «annullato sì o no». Quanto il cliente ha effettivamente inserito, quanto resto la macchina ha davvero erogato, quanto non è riuscita a erogare, e con quale identificativo ha registrato il movimento: niente di tutto questo attraversa il confine, e niente viene mai scritto da nessuna parte.

Di conseguenza il denaro sta in due posti che non si parlano. **Fisicamente** è dentro la macchina. **Contabilmente** Giano registra l'ordine con l'importo *nominale* — la somma delle righe — e un unico incasso in contanti di pari valore. I due numeri coincidono solo se tutto è andato bene.

Quando non va bene, il disallineamento è silenzioso. Se la macchina non riesce a erogare tutto il resto, l'ordine si finalizza ugualmente per intero, l'operatore preme OK e i soldi in eccesso restano dentro la macchina **senza che nemmeno una riga di registro lo annoti**. Se il cliente annulla dal pannello della macchina, la finestra di Giano resta aperta a oltranza. Se cade la rete, l'interrogazione si spegne e non riprende più. In tutti e tre i casi l'ordine si chiude come se fosse stato pagato.

E a incasso concluso Giano apre comunque il cassetto contanti, anche se i soldi non ci sono passati.

1. Quanto è grande il pezzo VNE

Sessantanove file, circa 9.640 righe, più innesti dentro una ventina di file condivisi.

Gruppo	File	Righe	A che serve
Protocollo	45	2.587	la traduzione da e verso il linguaggio della macchina
Dispositivo	1	540	l'oggetto che Giano accende, spegne e interroga
Contratto di famiglia	5	137	il punto di innesto: che cosa Giano si aspetta da «una cassa automatica»
Servizio e ciclo di vita	1	256	chi crea i dispositivi all'avvio e li tiene vivi
Configurazione	1	34	che cosa si salva di ogni macchina
UI di amministrazione	13	4.576	registrare, configurare, provare
UI di vendita	3	1.510	la finestra che vede il cliente mentre paga

Due numeri da tenere a mente perché cambiano le priorità:

- **Il gruppo del protocollo sparisce interamente** con il pagAmico: 2.587 righe che vengono sostituite dalla nostra DLL. Sono quarantacinque file, quasi tutti oggetti-risposta da venti righe l'uno.
- **La finestra di prova hardware da sola è 3.056 righe**, cioè il sessantasette per cento di tutta l'interfaccia di amministrazione, e serve solo a collaudare.

Il **contratto di famiglia** è il gruppo più piccolo — centotrentasette righe — ed è quello che decide tutto il resto: **nove dei ventiquattro membri dell'interfaccia descrivono la macchina VNE, non una famiglia di macchine.**

2. Che cosa cambia col pagAmico

Non manca quasi nessun comando. Il lavoro è **tradurre un modello in un altro**.

	VNE (oggi)	pagAmico
Trasporto	HTTP, una richiesta per operazione	socket TCP persistente
Stato della transazione	sulla macchina , con un identificativo	nel client , in memoria
Avanzamento	si interroga ogni mezzo secondo	la macchina lo spinge da sola
Concorrenza	libera: più thread, più richieste	un comando alla volta
Transazioni aperte	una coda, elencabile	una sola; se cade la rete la macchina accetta la riconnessione dallo stesso IP, ma non si sa ancora se l'esito arriva sul nuovo socket
Dopo un riavvio	si enumerano i pendenti e si annullano	resta solo l'ultimo messaggio trasmesso, e solo a incasso chiuso: durante IN la macchina accetta solo AN e CM

Conseguenze concrete sul contratto: **tre membri non hanno alcun corrispondente** — interrogazione, elenco pendenti, chiusura sessione — e contando anche le due modalità di rifornimento e lo stato cassa completo, i membri da **eliminare** sono sei. Il thread di interrogazione non si adatta: si riscrive.

Il punto che fa più male, ed è contabile. Oggi Giano annulla chiedendo alla macchina di **trattenere** il denaro già inserito, e registra l'operazione come **annullata**: nessuna vendita, i soldi restano nel dispositivo. Il comando pagAmico che compie lo stesso gesto fisico significa **incasso accettato e chiuso**. Stessa azione sul contante, esito contabile opposto. Non esiste una traduzione neutra: è una decisione, e va presa prima di scrivere codice.

La compatibilità non è un problema

Sembrava il punto più spinoso — una libreria moderna dentro un progetto in C# 7.3 con un formato di progetto vecchio — e non lo è:

- la libreria dichiara già il target giusto e **la DLL è compilata**;
- le sue dieci dipendenze sono **già tutte in Giano alla versione identica**: zero pacchetti nuovi, zero configurazioni da toccare;
- il vincolo del linguaggio riguarda il codice **di Giano**, non un assembly referenziato.

Quindi: **la DLL va fra le referenze esterne**, non ricompilata come progetto. Ricompilarla in C# 7.3 costerebbe la rimozione di oltre un centinaio di annotazioni di nullabilità sulla superficie pubblica — che sono l'unica documentazione formale di che cosa può essere nullo in un'interfaccia che maneggia denaro — per un beneficio funzionale nullo.

3. Il precedente: l'abbiamo già fatto tre mesi fa

Questa è la parte che dà fiducia alla stima, ed è il pezzo che vale la pena raccontare al telefono.

Fra l'8 giugno e l'1 luglio è stata aggiunta una quarta marca di cassetto contanti, il cassetto OPOS dei terminali AURES. Il percorso è documentato e leggibile:

Data	Cosa
8 giugno	il piano di implementazione, scritto prima di toccare codice
26 giugno	l'aiutante a 32 bit — 3 file, 378 righe
26 giugno	gli esiti della prova sull'hardware reale, congelati nel piano
29 giugno	tutto il lato Giano: 33 file, 2.732 righe aggiunte, 41 tolte, in un commit solo
1 luglio	validazione completa sulla macchina

Che cosa insegna

L'impalcatura per aggiungere una marca esiste ed è collaudata. Enum, quattro tabelle di traduzione, record di configurazione, tabella e CRUD, procedura guidata di registrazione, sincronizzazione remota, navigazione, pagina hub, pagina elenco, finestra di modifica, stringhe in tre lingue, elenco file nel progetto: tutto questo è la stessa sequenza di gesti, sugli stessi file, con le stesse convenzioni. È circa **due terzi** di quel commit, ed è codice che si scrive guardando il precedente.

Due dettagli che riguardano direttamente il pagAmico:

- **La tabella nuova non ha richiesto una migrazione.** Nel codice c'è il commento esplicito: la tabella è stata aggiunta dopo il rilascio iniziale, e viene creata se manca, in modo idempotente. È esattamente il modello da riusare.
- **Il pezzo fuori sagoma è costato più di tutto il resto.** Il driver AURES è a 32 bit e Giano gira a 64: la soluzione è stata un eseguibile separato, e fra il piano e l'aiutante funzionante sono passate quasi **tre settimane**. Le 2.732 righe del lato Giano sono venute dopo, in un colpo solo.

Dove il precedente NON aiuta

Tre differenze, e vanno dette:

1. **L'interfaccia del cassetto era già un'astrazione di famiglia; quella della cassa automatica non lo è.** Il cassetto aveva già tre marche vive quando è arrivata la quarta: l'interfaccia era stata *ricavata* da più dispositivi, e solo due dei suoi diciotto membri sono specifici. Per la cassa automatica invece la seconda marca è prima **un'estrazione** e poi un'aggiunta.
2. **Il cassetto fa una cosa sola e non tiene denaro.** Aprire è sincrono, immediato, senza conseguenze contabili.
3. **La vendita non era toccata; qui lo è.** Quel commit non contiene un solo file sotto le pagine di vendita. Qui va riscritta una finestra da 981 righe costruita attorno a un modello di interrogazione che nel pagAmico non esiste.

La somiglianza vera, e utile. In entrambi i casi il rischio non sta nell'impalcatura ma nell'**adattatore di forma** — l'aiutante a 32 bit lì, il ponte fra il modello a spinta del pagAmico e il modello a thread di Giano qui. Nel cassetto OPOS quel pezzo è stato scritto e provato **per primo, da solo, contro l'hardware vero**, prima di toccare una riga di Giano. È esattamente l'ordine da ripetere.

4. Quanto lavoro è

Giornate uomo di uno sviluppatore che conosce Giano. Taratura: il cassetto OPOS completo — aiutante, lato Giano e collaudo — è valso circa **otto-dieci giornate**, per un dispositivo che fa una cosa sola e non tocca la vendita.

	Blocco	Giornate	Natura
0	referimento alla DLL nel progetto	0,25	meccanico
A	registrazione hardware	0,5	meccanico
B	database (record, tabella, CRUD, integrità, sincronizzazione)	1 – 1,5	meccanico, con una decisione
C	adattatore fra la libreria e Giano	3 – 5	ricerca
D	dispositivo e rifacimento del contratto di famiglia	2 – 3	decisioni
E	interfaccia di amministrazione	2 – 2,5	meccanico
F	flusso di vendita	4 – 6	decisioni contabili
G	prove: simulatore, poi macchina reale	2 – 3	misto, più tempo macchina
	Totale sostituzione «alla pari»	15 – 22	
F+	<i>opzionale</i> : chiudere il buco contabile e conservare l'identificativo	+3 – 5	decisioni, tocca il database delle vendite

I blocchi meccanici: circa cinque giornate, rischio basso

I blocchi 0, A, E e la scrittura di B sono clonazione guidata dal precedente. Su questo la stima è solida: il cassetto OPOS ha prodotto 2.732 righe della stessa natura in un commit.

Una cosa da non fare. Clonare la finestra di prova hardware, che nel VNE è 3.056 righe. Il cassetto OPOS ha risolto il collaudo con **un pulsante** sulla pagina elenco. Se qualcuno propone di replicare la finestra di prova, sono tre-cinque giornate in più per una cosa che serve una volta.

I blocchi con decisioni: undici-diciassette giornate, e la stima non stringe

C — l'adattatore. È il pezzo senza precedente in casa. Deve trasformare un'interfaccia asincrona a spinta in un'operazione bloccante con avanzamento e annulla, e prima della risposta di PayPrint sembrava dover convivere con quattro vincoli nuovi: il turno unico che tiene la transazione, il predicato di chiusura che si accontenta di qualunque risposta testuale, la pausa minima fra comandi, la connessione persistente da riaprire a mano. Dopo la risposta i primi due si sono rivelati limiti della libreria, e sono corretti dall'11 settembre: CM e AN a incasso aperto passano per una via laterale invece di mettersi in coda dietro l'IN, e il predicato — il difetto D3 di esito-risposta-payprint.md — lavora in due fasi. La pausa, per il fornitore, non serve se i comandi terminano con CR o CR+LF; ma oggi la libreria non manda terminatore, e la pausa resta finché il CR non è provato sul simulatore e una raffica di comandi con CR non regge su una macchina vera. E se la rete cade a incasso aperto, la macchina accetta la riconnessione dallo stesso IP — se poi l'esito arrivi sul nuovo socket, non si sa ancora.

Non era stimabile sotto le tre giornate finché non si sapeva se esisteva una via supportata per chiudere-e-trattenere durante un incasso, e che cosa conteneva esattamente un messaggio di avanzamento. Erano le domande 2.1 e 2.7 a PayPrint, e la risposta dell'11 settembre le chiude entrambe:

durante un incasso la macchina accetta CM, e i messaggi di avanzamento arrivano a ogni aggiunta di contante e sono cumulativi. La condizione è caduta: l'adattatore si può disegnare.

La tabella lascia comunque il blocco C a 3 – 5 giornate: la risposta toglie l'incognita che bloccava il disegno, ma aggiunge lavoro nella nostra libreria, che nessun blocco della tabella comprende e che va stimato a parte. I due limiti qui sopra e i difetti D1 e D3 sono fatti (11 settembre); resta D2 — per cui serve prima il keepalive regolato — e il resto è elencato in ordine nel §3 di `esito-risposta-payprint.md`. Alcune risposte che servono all'adattatore (quanti frame seguono un CM, che cosa porta un AN con denaro dentro, che cosa arriva sul nuovo socket dopo una riconnessione) aspettano ancora la telefonata: sono nel §5 dello stesso documento.

D — il contratto. Sei membri si eliminano, uno si rimappa, due si rifanno. È lavoro di progettazione, non di battitura, e va fatto **prima** di E e F perché ne fissa le firme. Qui il precedente non aiuta: l'interfaccia del cassetto non era stata toccata.

F — il flusso di vendita. Le 981 righe della finestra di pagamento si riscrivono, non si adattano: sparisce il thread di interrogazione, sparisce il terzo thread di sorveglianza, spariscono i flag booleani usati come mutex fra cinque thread. E vanno prese quattro decisioni contabili che nessuno può prendere leggendo il codice:

- annullo con denaro dentro: si trattiene o si restituisce? Trattenere con CM può voler dire trattenere più del dovuto: sul simulatore 470 € su 80 richiesti, senza resto, e se la macchina vera renda il resto è la seconda domanda da fare in telefonata (§5 di `esito-risposta-payprint.md`). Per rendere l'eccedenza serve PA, che per il fornitore non dovrebbe essere abilitato di default.
- il buco del resto non erogato si chiude o si accetta?
- l'identificativo del movimento si conserva sulla finalizzazione?
- senza timeout — il fornitore dice di non metterlo, e quello di cinque minuti della libreria allo scadere abbandona l'incasso aperto (difetto D2) — chi chiude un incasso che non finisce: l'operatore dalla finestra, con AN o CM, o la chiusura forzata dal pannello, che arriva come AN?

Le quattro-sei giornate valgono per l'ipotesi minima: si traduce il comportamento attuale il più fedelmente possibile e **si allarga il confine fra la finestra e il resto dell'applicazione**, da un booleano a un oggetto di esito. Se si decide anche di chiudere il buco contabile si aggiungono le tre-cinque giornate della riga in fondo, che toccano il database delle vendite — cioè la parte del programma dove non si torna indietro.

G — le prove. L'ordine giusto è quello del cassetto OPOS: **prima l'adattatore da solo contro il simulatore, poi contro la macchina vera, e solo dopo il resto di Giano**. Le due-tre giornate non includono l'attesa dell'hardware né il tempo per far rispondere PayPrint — che sul precedente sono state la parte più lunga del calendario. La prima risposta di PayPrint è arrivata l'11 settembre, con l'offerta di collegarci alla sua macchina per le prove; restano le domande da fare in telefonata, elencate nel §5 di `esito-risposta-payprint.md`.

5. La sintesi, da dire a voce

L'impalcatura per aggiungere una seconda marca c'è già ed è collaudata: l'abbiamo fatto tre mesi fa con il cassetto OPOS, in un commit da trentatré file. Quella parte sono quattro-cinque giornate e il rischio è basso. Il resto — il ponte verso la nostra libreria del pagAmico e la riscrittura della finestra di pagamento — sono altre undici-diciassette giornate, e non è impalcatura: è la parte dove si decide che cosa succede ai soldi quando qualcosa va storto. Su quella ci sono **quattro domande contabili** che deve decidere il committente, e finché non hanno risposta la stima non stringe. Le **due domande tecniche** che bloccavano il progetto hanno avuto risposta da PayPrint: la macchina permette di chiudere un incasso trattenendo il denaro, quindi trattenere o restituire è davvero una scelta. Il lavoro tecnico che resta è soprattutto nella nostra libreria, dove la risposta ha fatto emergere tre difetti da correggere e da stimare a parte; e alcune domande restano da chiarire con PayPrint in telefonata, con prove sulla sua macchina.

E una cosa che vale più di tutte le altre messe insieme:

Sostituire il box senza allargare il confine della finestra di pagamento è lavoro sprecato. Il pagAmico offre *più* dati della cassa attuale — l'importo non erogato in un campo dedicato invece che da ricavare per sottrazione, e un identificativo di movimento che il dispositivo conserva e che si può riconciliare a posteriori. Ma finché da quella finestra esce un booleano, si perdono esattamente come si perdono oggi, e nessuno dei buchi contabili esistenti si chiude.

Il dettaglio riga per riga — quali file, quali righe, quali punti di estensione — sta in `analisi-giano-vne-vs-pagamico.md`. Le decisioni aperte stanno nel documento 5.